Step-by-Step Guide: Analysing Traffic with Wireshark (GUI and CLI)



Wireshark is the de facto standard for network packet analysis. This guide covers both interfaces: the graphical **Wireshark GUI** and the command-line **tshark** tool that ships with it. In the CREST CPIA lab, tshark runs inside the `cpia-forensics` analyst container; the GUI runs on a desktop with a display.

The worked examples below use the real CPIA lab capture at `/work/network.pcap` (also at `~/labs/cpia-labs/work/network.pcap` on the host). This capture contains the actual incident: an SSH brute force and HTTP reconnaissance from attacker host `172.19.0.4` against victim `172.19.0.3`.

© Billy Forrest @ https://training.forrest.one

1. Understand the two tools

Tool Interface Use `wireshark`	GUI	Interactive capture, protocol dissection, follow streams
`tshark`	CLI	Scriptable capture and analysis, offline pcap reading
`capinfos`	CLI	Metadata about a capture file
`mergecap`	CLI	Merge multiple capture files
`editcap`	CLI	Split / filter capture files by time or packet

Both `wireshark` and `tshark` use the **same display filter syntax**, so anything you learn in one transfers to the other.

---

2. Start a capture

GUI (wireshark)

1. Open the Capture panel (or `Ctrl+K`).
2. Select the interface you want to monitor (e.g. `eth0`).
3. Click the blue shark-fin button, or press `Ctrl+E` to begin.
4. Click the red stop button (or `Ctrl+E`) to end and keep the capture.

CLI (tshark)

List the available interfaces:

```
tshark -D
```

Capture live on an interface, writing to a file:

```
tshark -i eth0 -w /work/live.pcap
```

Capture with a limit (e.g. 100 packets) and stop:

```
tshark -i eth0 -c 100 -w /work/live100.pcap
```

> *In the CPIA lab you normally analyse an existing capture rather than capture live, because the lab traffic was already collected during the incident.*

---

3. Read an existing capture

GUI

File > Open, then select the `.pcap` file. The three panes appear:

- **Packet list** (top) - each captured frame on one row
- **Packet details** (middle) - the decoded protocol tree for the selected packet
- **Packet bytes** (bottom) - the raw hexadecimal and ASCII view

CLI

Read the capture and print a one-line summary per packet:

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

```
tshark -r /work/network.pcap
```

Count how many packets are in the file:

```
tshark -r /work/network.pcap 2>/dev/null | wc -l
# 450 packets
```

Get capture file metadata:

```
capinfos /work/network.pcap
```

---

4. Understand what is in the capture

Before filtering, see which protocols dominate. The protocol hierarchy shows every protocol layer and how much data it carries:

```
tshark -r /work/network.pcap -q -z io,phs 2>/dev/null
```

For the CPIA lab capture this shows:

```
frame                                    frames:440
  eth                                  frames:440
    ip                                 frames:434
      tcp                              frames:434
        ssh                            frames:99     # brute force
        http                           frames:36     # reconnaissance
```

Which tells you the story at a glance: an SSH attack and HTTP probing.

See the two hosts involved (the attacker and victim):

```
tshark -r /work/network.pcap -q -z endpoints,ip 2>/dev/null
```

---

5. Filter with display filters

Display filters narrow what you see to packets matching a condition. They work identically in the GUI filter bar and in `tshark -Y`.

Common filters

```
ip.addr == 172.19.0.4          # all traffic to/from the attacker
ip.src == 172.19.0.4           # traffic FROM the attacker
ip.dst == 172.19.0.3           # traffic TO the victim
tcp.port == 22                 # SSH
tcp.port == 80                 # HTTP
http.request                   # only HTTP request packets
http.response                  # only HTTP responses
ssh                            # all SSH traffic
```

CLI examples (using -Y)

```
# all packets involving the attacker
tshark -r /work/network.pcap -Y "ip.addr == 172.19.0.4" 2>/dev/null

# only HTTP requests
tshark -r /work/network.pcap -Y "http.request" 2>/dev/null
```

That HTTP filter reveals the attacker's reconnaissance path:

© Billy Forrest @ https://training.forrest.one

```
226  4.03  GET /admin          HTTP/1.1
238  4.14  GET /admin/index.php HTTP/1.1
251  4.25  GET /config.php     HTTP/1.1
263  4.35  GET /backup         HTTP/1.1
275  4.46  GET /backup.zip     HTTP/1.1
287  4.57  GET /phpmyadmin/    HTTP/1.1
299  4.68  GET /wp-login.php   HTTP/1.1
311  4.79  GET /server-status  HTTP/1.1
```

This is textbook reconnaissance: probing admin panels, backup files, database front-ends, and status pages.

GUI

Type the same expression into the green filter bar (top of the window) and press `Enter`. Use the book icon to save reusable filters.

---

6. Combine and refine filters

Use Boolean operators to build precise queries:

```
ip.addr == 172.19.0.4 && tcp.port == 80       # attacker's HTTP traffic
http.request && ip.src == 172.19.0.4          # requests sent by attacker
tcp.port == 22 || tcp.port == 80              # SSH or HTTP
!(arp)                                        # everything except ARP
```

```
tshark -r /work/network.pcap -Y "ip.addr == 172.19.0.4 && tcp.port == 80" 2>/dev/null
```

---

7. Follow a conversation / stream

GUI

Right-click a packet > **Follow** > **TCP Stream** (or `Ctrl+Alt+Shift+T`). This reassembles and shows the full application-layer conversation, exactly what the attacker typed or downloaded. The best way to see a whole HTTP request/response as one readable block.

CLI

Export a conversation as a readable stream:

```
tshark -r /work/network.pcap -q -z follow,tcp,ascii,1 2>/dev/null
```

The `1` is the TCP stream index (start at 0). Use `follow,tcp,raw` for the hex/ASCII form.

---

8. Extract statistics for a report

Statistics make findings reportable and quantifiable.

Top talkers

```
tshark -r /work/network.pcap -q -z endpoints,ip 2>/dev/null
```

TCP conversations

```
tshark -r /work/network.pcap -q -z conv,tcp 2>/dev/null
```


© Billy Forrest @ https://training.forrest.one

HTTP requests (tabular view for reports)

The cleanest way to list every HTTP request with its source and URI, using field extraction:

```
tshark -r /work/network.pcap -Y "http.request" \
       -T fields -e frame.number -e ip.src \
       -e http.request.method -e http.request.uri 2>/dev/null
```

This prints one row per request - ideal for a report table:

```
226  172.19.0.4  GET  /admin
238  172.19.0.4  GET  /admin/index.php
251  172.19.0.4  GET  /config.php
263  172.19.0.4  GET  /backup
275  172.19.0.4  GET  /backup.zip
287  172.19.0.4  GET  /phpmyadmin/
299  172.19.0.4  GET  /wp-login.php
311  172.19.0.4  GET  /server-status
323  172.19.0.4  GET  /robots.txt
335  172.19.0.4  GET  /files/1.txt
347  172.19.0.4  GET  /etc/passwd
359  172.19.0.4  POST /login
```

HTTP request statistics

Aggregate the requests per host and URI with the `http_req,tree` statistic:

```
tshark -r /work/network.pcap -q -z http_req,tree 2>/dev/null
```

This reveals request patterns, including the `/pixel.gif?id=beacon&c=N` C2 beaconing observed in the lab capture.

Save a filter-based export for evidence

```
tshark -r /work/network.pcap -Y "http.request" -w /work/http-requests.pcap 2>/dev/null
```

---

9. Search for indicators of compromise (IoC)

Use tshark to scan the whole capture for suspicious strings, such as a command injection payload, a username from a brute force, or a known-bad IP:

```
# search packet payloads for a string
tshark -r /work/network.pcap -Y "frame contains \"admin\"" 2>/dev/null

# search for a specific IP
tshark -r /work/network.pcap -Y "ip contains \"172.19\"" 2>/dev/null

# search raw HTTP for a credential or path
tshark -r /work/network.pcap -Y "http contains \"passwd\"" 2>/dev/null
```

In the GUI, use **Edit > Find Packet** (or `Ctrl+F`) with the filter bar to search the same way.

---

10. The GUI workflow at a glance

1. **Open** the capture (File > Open).
2. **Colour** suspicious traffic: View > Coloring Rules (SSH brute force, HTTP errors, etc.).
3. **Filter** to the attacker: `ip.addr == 172.19.0.4`.
4. **Follow** the TCP stream to read the full request/response.

© Billy Forrest @ https://training.forrest.one

5. **Add a comment** (right-click > Packet Comment) to annotate evidence.
6. **Export** your evidence: File > Export Packet Dissections, or keep a filtered copy of the pcap.

Quick reference

```
# list interfaces
tshark -D

# live capture
tshark -i eth0 -w /work/live.pcap

# read a file (one line per packet)
tshark -r /work/network.pcap

# count packets
tshark -r /work/network.pcap 2>/dev/null | wc -l

# protocol hierarchy
tshark -r /work/network.pcap -q -z io,phs 2>/dev/null

# filter by host and port
tshark -r /work/network.pcap -Y "ip.addr == 172.19.0.4 && tcp.port == 80" 2>/dev/null

# only HTTP requests
tshark -r /work/network.pcap -Y "http.request" 2>/dev/null

# endpoints (top talkers)
tshark -r /work/network.pcap -q -z endpoints,ip 2>/dev/null

# TCP conversations
tshark -r /work/network.pcap -q -z conv,tcp 2>/dev/null

# follow a TCP stream (index 1)
tshark -r /work/network.pcap -q -z follow,tcp,ascii,1 2>/dev/null

# list HTTP requests as a table
tshark -r /work/network.pcap -Y "http.request" -T fields -e frame.number -e ip.src -e http.request.method -e ht

# HTTP request statistics by host/URI
tshark -r /work/network.pcap -q -z http_req,tree 2>/dev/null

# search payloads for a string
tshark -r /work/network.pcap -Y "frame contains \"admin\"" 2>/dev/null

# export a filtered subset
tshark -r /work/network.pcap -Y "http.request" -w /work/http-requests.pcap 2>/dev/null
```

Pitfalls

- **GUI vs CLI binaries.** In the CPIA analyst container only the CLI (`tshark`) is installed by default. The GUI (`wireshark`) needs a desktop environment and a display. On a headless server use tshark.
- **Root warnings.** tshark prints "Running as user root ... dangerous" - this is informational. It still works.
- **Noise from `2>/dev/null`.** Some tshark versions emit SSH dissector warnings for unusual key-exchange algorithms. Redirect stderr (`2>/dev/null`) to keep output clean, or use `-q` for statistics.
- `tshark -q` **vs** `-Y`**.** `-q` suppresses per-packet output (used with `-z` statistics). `-Y` applies a display filter to the packet list. They do different jobs; combine them as shown above.

© Billy Forrest @ https://training.forrest.one

- **Stream index in Follow.** `follow,tcp,ascii,N` starts at stream 0. Use the stream number you see in the GUI packet details if matching a specific conversation.
- **Time zones.** pcap timestamps are usually UTC. Note the zone when you quote event times in a report.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one